```
data['sales'].plot(title='Sales Over Time', figsize=(10, 6)) plt.show()

# Step 3: Train-Test Split
train = data[:'2022-12-31']
test = data['2023-01-01':]

# Step 4: Train SARIMA Model
model = SARIMAX(train['sales'], order=(1, 1, 1), seasonal_order=(1, 1, 1, 12)) results = model.fit()

# Step 5: Forecast
forecast = results.get_forecast(steps=len(test)) forecast_index = test.index
forecast_values = forecast.predicted_mean confidence_intervals = forecast.conf_int() # Step 6: Evaluate
mae = mean_absolute_error(test['sales'], forecast_values) mse = mean_squared_error(test['sales'], forecast_values) print(f"MAE: {mae}, MSE: {mse}")

# Step 7: Visualize Predictions plt.figure(figsize=(10, 6))
plt.plot(train['sales'], label='Training Data') plt.plot(test['sales'], label='Actual Sales', color='orange')
plt.plot(forecast_index, forecast_values, label='Forecast', color='green')
plt.fill_between(forecast_index, confidence_intervals.iloc[:, 0], confidence_intervals.iloc[:, 1], color='green', alpha=0.2) plt.legend()
plt.title('Sales Forecasting')
plt.show()
```

Project 8: Object Detection in Images

Objective:

To identify and locate objects within images, supporting applications that require detailed object localization, such as autonomous vehicles, security systems, and retail analytics.

Detailed Plan:

1. Dataset Selection:
    - Use publicly available datasets like:
        - **COCO (Common Objects in Context):** Contains labeled images with bounding boxes for various objects.
        - **Pascal VOC (Visual Object Classes):** Provides annotations for 20 object categories.
    - Optionally, collect domain-specific data and annotate using tools like LabelImg for custom applications.